1

Software Development 1 · Week 1

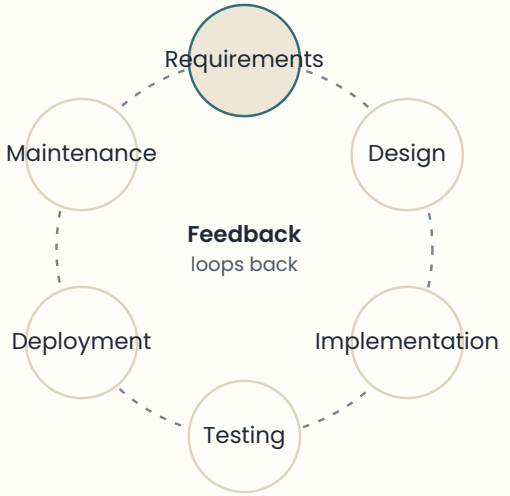
Introduction to Software Engineering Management

Software engineering is a discipline about delivering and maintaining software, not only writing it.

- Most project problems come from communication and planning gaps.
- Models let teams discuss and check a system before building it.

> **Think of it like this.** Building a house needs blueprints before bricks. Software needs plans and models before code, because changing a wall on paper costs far less than tearing it down.

A process model is a simplified description of how real teams work.



> **Think of it like this.** Think of a cooking class. Waterfall is following a recipe from start to finish without tasting. Iterative work is tasting the sauce after each step and adjusting the next one.

A requirement must be checkable: someone should be able to say "met" or "not met".



> **Think of it like this.** A tailor measures twice and cuts once. Requirements are the measurements: if they are wrong, no amount of careful sewing (coding) will make the suit fit.

A model is useful when it is simpler than the system but still correct about what matters.



> **Think of it like this.** A city map is a model. It ignores the colour of each house (abstract) but gets the streets right (precise), so you can plan a route before walking (predictive and cheap).

Configuration items include everything that makes up the product, not only code.

- SCM answers: which version, which change, approved by whom, and in which baseline.

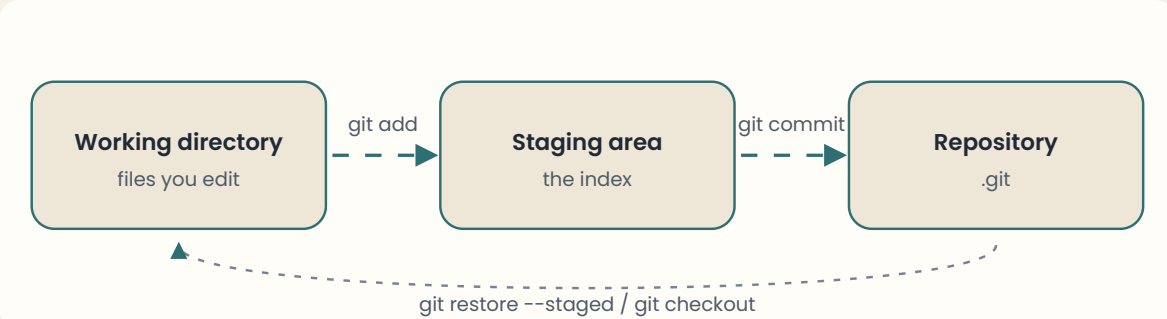
> **Think of it like this.** A library keeps a catalogue of every edition and who borrowed it. SCM does the same for a product: every item, version, and change has a record you can check.

A VCS stores history, supports recovery, and shows who changed what and when.

- Git is distributed; GitHub is a hosting service for Git repositories.
- One commit should hold one logical change with a clear message.

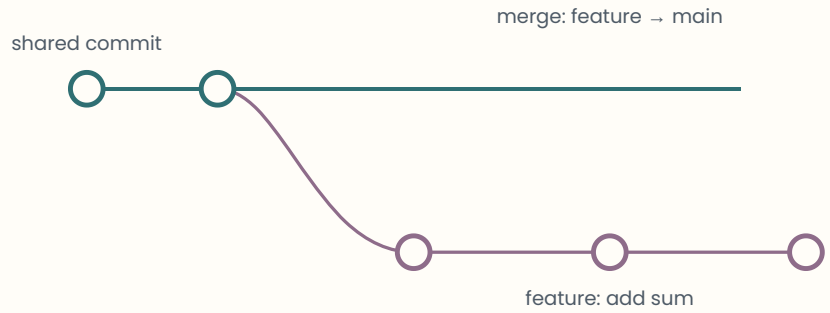
> **Think of it like this.** Version control is like a video game with save points. You can go back to any save, see what changed between two saves, and let friends play from their own copy.

Edit in the working directory, stage with git add, and record with git commit.



> **Think of it like this.** Git is a photo studio. The working directory is the set, staging is choosing which photos go in the album, and commit is printing the album page. A branch is a second album you can build without touching the first.

origin is the conventional name for the remote you cloned or connected.



Each dot is a commit. Branches are pointers; merging combines history.

> **Think of it like this.** A pull request is like a draft essay sent to a classmate before handing it in. The reviewer does not write it for you, but catches mistakes while they are cheap to fix.

Step by step

Put the lifecycle in order

1. Requirements analysis
2. Design
3. Implementation
4. Testing
5. Deployment
6. Maintenance

Check yourself

1. Which item is a configuration item in software configuration management?

A) A requirements document B) The office coffee machine C) A personal calendar

2. Which statement about Git is correct?

A) Git is a distributed version control system; every clone holds the full history. B) Git cannot record a commit without an internet connection to GitHub. C) Git and GitHub are two names for the same tool.

3. In an iterative and incremental process, what does the difference between two successive baselines represent?

A) An increment B) A risk register C) The final delivery only

Answers are in the speaker notes.

Practise next

Labs

- Initialise a repository and make your first commit
- Read history and work on a branch
- Resolve a merge conflict
- Connect to a remote and simulate a review

On the site

- Git Playground